

FinAssist — AI Customer Support Agent for Fintech

Core Concept

A customer-facing support agent for a fintech company (digital wallet / neobank style) that answers account questions, performs safe read-only actions, and knows exactly when to stay silent and escalate to a human — instead of guessing.

Backend Architecture (reusing your Nexus skeleton)

Router (the brain)

Same pattern as Nexus — one LLM call decides which of 3 modes handles the query, with the same RAG-first, honesty-first system prompt discipline you already built and tested.

Mode 1 — use_rag: Knowledge Base Search

Searches real support content, chunked and stored in Chroma:

- Fee schedules (transfer fees, ATM fees, subscription tiers)
- KYC/verification requirements
- Dispute & chargeback policy
- Refund policy
- Terms of service excerpts
- FAQ ("how do I reset my PIN", "how long do transfers take")

Mode 2 — use_tool: Account Actions (read-only, safe)

Real functions — since it's a portfolio project, these hit a **mock account database** you create (a small SQLite/JSON file with fake users), not a real bank:

- check_transaction_status(transaction_id)
- check_account_balance(user_id) — gated behind a verification check
- calculate_refund_eligibility(transaction_id)
- check_verification_status(user_id)

Mode 3 — escalate_to_human (replaces delegate_research)

Triggers when:

- The query needs account changes (not just reads) — e.g., "close my account," "dispute this charge officially"
- The query is emotionally sensitive — fraud reports, financial hardship
- The RAG search and tools both come up empty

```
def escalate_to_human(query, reason):
```

```
    ticket_id = create_ticket(query, reason)
```

```
return f"Escalated to human support (ticket #{ticket_id}). Reason: {reason}"
```

Compliance Guardrail Layer (the differentiator)

Baked into the router's system prompt, same discipline as Nexus's honesty rule:

- Never state an account balance without a verified user_id
 - Never speculate on investment/financial advice — redirect to human/advisor
 - Never confirm or deny sensitive actions (like "was my account flagged for fraud") — always escalate those
 - Always cite source: "per our fee schedule" (RAG) vs "per your account" (tool) vs "this needs human review" (escalation)
-

Frontend — what to actually build, and why each piece matters

Keep this simple (React or even a clean HTML/JS single page) — the backend is the substance, the frontend just needs to make the agent's reasoning visible, since that transparency IS a feature here, not just polish.

1. Chat interface (core)

Standard message thread — user types, agent responds. Nothing fancy needed structurally.

2. "Source" badge on every agent answer — this is the important one

Each response visibly tags where the info came from:

-  **From Knowledge Base** (RAG) — with a link/expand to the actual policy excerpt
-  **From Your Account** (tool call) — shows which action was run
-  **Escalated to Human** — shows a ticket number This single UI feature makes your compliance/honesty work *visible* rather than buried in logs — huge for a demo, since an interviewer can see the routing decision in real time.

3. Mock login / account selector

Since there's no real bank, let the user pick a demo persona ("Login as Test User A — verified" / "Test User B — unverified"). This lets you *demonstrate* the verification-gating guardrail live: ask the same balance question as both users, show one gets a real answer and the other gets "please verify your identity first."

4. "Agent reasoning" toggle (optional but strong for interviews)

A collapsible panel showing the router's tool-call sequence — same reasoning_content you've been reading in your terminal this whole project, just surfaced in the UI. Toggle it on during a demo to show *why* the agent made its routing decision — this turns your debugging transcripts into a live product feature.

5. Escalation ticket view

A simple panel listing "tickets" created when the agent escalated — shows the query, the reason, and a fake "pending human review" status. Proves the escalation path isn't just a stubbed string, it's a real (if minimal) workflow.

Why this scope is right — not too much, not too little

- Everything backend reuses your proven Nexus architecture — router, RAG, tool-agent, honesty guardrails
- The NEW work is: mock account data/DB, support-specific tools, escalation logic, and the frontend
- The frontend is deliberately scoped to make backend decisions *visible*, not to be a polished consumer app — that's the right prioritization for a portfolio piece being evaluated by engineers, not designers

Want to start with the backend — designing the mock account schema and the use_tool functions — or the frontend skeleton first?

Great questions — let's go through each concretely.

What does the escalation ticket actually do?

Right now, conceptually, it's a structured record created when the agent decides it can't/shouldn't handle something itself:

```
def escalate_to_human(query, reason, user_id=None):
```

```
    ticket = {
        "ticket_id": generate_id(),
        "user_id": user_id,
        "query": query,
        "reason": reason,
        "status": "pending",
        "created_at": datetime.now(),
        "conversation_context": messages[-5:] # last few turns, so the human isn't starting blind
    }
```

```
    save_ticket(ticket) # store in a JSON file or SQLite table
```

```
    return f"Escalated to human support (ticket #{ticket['ticket_id']})"
```

It's stored somewhere persistent (SQLite is cleaner than JSON for this), and its whole purpose is to hand off context to a human — not just log that escalation happened, but give the human agent enough to actually act without re-asking the customer everything.

Two connected UIs — yes, and this is a genuinely good addition

This is a strong idea because it completes the product loop — right now the "escalation" is a dead end (a ticket gets created and nothing else happens). Two UIs make it real:

UI 1 — Customer chat (what we discussed): the agent, source badges, escalation confirmation **UI 2 — Agent dashboard**: a simple queue view for the human support side

- List of pending tickets (query, reason, timestamp, customer context)
- Click a ticket → see the full conversation that led to escalation
- A reply box → human types a response → gets saved back and (ideally) shown to the customer as "a human has replied"
- Mark ticket resolved/closed

This turns the project from "an agent that punts hard questions" into "a full human-in-the-loop support system" — genuinely more complete and more impressive, since it shows you thought about the *whole* workflow, not just the AI part.

Is this single-agent or multi-agent, like Nexus?

Same multi-agent pattern as Nexus — this is a direct continuation of that architecture, not a different one:

- **Router** (top-level LLM decision maker) → decides RAG vs account-tool vs escalate
- **RAG** → its own retrieval logic (not really an "agent" itself, more a data-lookup service)
- **Tool sub-agent** → could be a single LLM call for simple account lookups, or a full reasoning loop like your weather/calculator agent if actions get more complex (e.g., "check my last 3 transactions and tell me which one might be the disputed charge")
- **Escalation** → not really an "agent" here, more a deterministic action — no need for it to reason, just create a ticket

So: yes, same "one decision-maker LLM routing to specialized handlers" pattern you already built and proved works.

The verification/login question — this is an important design decision

Yes — **enforce it properly, don't let the LLM decide**. This should NOT be something the agent "reasons about" — it should be a hard, code-level gate before the agent ever runs, otherwise you risk exactly the kind of prompt-injection/social-engineering failure real fintech bots need to avoid.

```
def check_account_balance(user_id, session_verified: bool):
```

```
    if not session_verified:
```

```
        return {"error": "verification_required", "message": "Please verify your identity to view account details."}
```

```
    return get_balance_from_db(user_id)
```

The **frontend** enforces this at the login/session level: "Test User B" is deliberately set up as `verified=False` in your mock data, so ANY account-specific tool call for that user returns the verification-required response — regardless of how the question is phrased. This is a great thing to demo: same question, two different logged-in users, two different (correct) outcomes. It's also a legitimate security-pattern talking point for an interview: "sensitive actions are gated at

the tool layer, not left to the LLM's judgment" — that's the right way to design this in a real system.

If unverified, the UI could show a "Verify your identity" prompt/redirect rather than just an error string — nice, cheap UX touch.

Other features worth adding (optional, pick based on time budget)

- **Sentiment/urgency detection** — flag escalations as high-priority if the user seems frustrated (based on message tone) — genuinely useful and demonstrable
- **Multi-turn account context** — the agent remembers which account/transaction was discussed earlier in the conversation, not just per-message
- **Audit log** — every tool call and RAG lookup logged with timestamp, useful for a "compliance" angle (fintech loves audit trails)
- **Confidence/uncertainty display** — show when the agent is less sure vs. confident, not just binary "answered" vs "escalated"

Suggested priority order

1. Core router + RAG + account tools + hard-gated verification (the safety-critical piece)
2. Escalation + ticket creation
3. Two UIs (customer + human dashboard)
4. Pick 1-2 "other features" above if time allows — audit log is probably the best ROI for a fintech-flavored portfolio piece

Want to start scoping the mock account database schema and the verification-gated tools next, since that's the foundation everything else sits on?